

ChargeGrid Intelligence

Integração, automação e eficiência para recarga elétrica

RELATÓRIO TÉCNICO DO PROTÓTIPO SIMULADO

Pensamento Computacional e Automação com Python
21 de setembro de 2026

Uma plataforma que transforma recarga em decisão energética

O **ChargeGrid Intelligence** integra sessões de recarga, gestão de demanda, prioridade solar, dados, cobrança e indicadores ambientais em um monólito modular demonstrável. Nesta Sprint 3, a integração é comprovada por um cenário automatizado e reproduzível, com componentes físicos substituídos por provedores simulados explicitamente identificados.

DEMONSTRAÇÃO INTEGRADA: sessão → alocação → solar/rede → leitura → alerta → billing → ESG

Entregável	Evidência no protótipo
Integração	React → API FastAPI → serviços → SQLAlchemy/PostgreSQL
Automação	Seed idempotente, roteiro HTTP e ticks manuais determinísticos
Eficiência	Limite de 60 kW da rede e rateio Equal Share
Sustentabilidade	Prioridade solar e cálculo de CO ₂ evitado
Qualidade	179 testes backend, 14 frontend, lint, tipos e build aprovados no ensaio registrado

Escopo e transparência

As grandezas apresentadas são **simuladas**. Não houve medição em carregador, inversor ou medidor físico. Não há OCPP/Modbus, agendador de ticks ou modelo de ML treinado nesta entrega; o risco de pico preditivo permanece para fase posterior. Essa delimitação preserva a rastreabilidade técnica do MVP.

Equipe 3 - FIAP x GoodWe

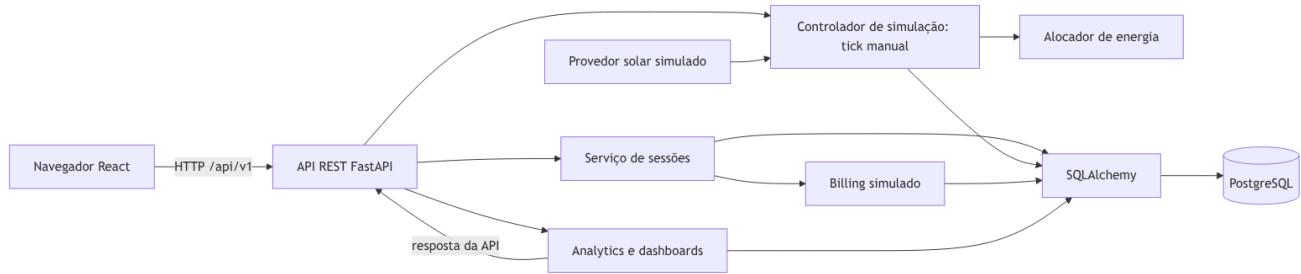
Nome completo	RM
Bernardo Zauza Amorim	568808
Bruno Almeida de Oliveira	572648
Gabriel Góes Nunes Pereira	571735
Guilherme Vinciguerra Carvalho	571951
Marcos Peterson Martins Pereira	573857
Matheus Jorge Santana	574166

Problema abordado

Quando quatro veículos solicitam 20 kW cada, a demanda chega a 80 kW, acima dos 60 kW disponíveis da rede. O sistema precisa limitar a importação, dividir potência de forma previsível, aproveitar solar antes da rede e registrar consequências operacionais, financeiras e ambientais.

Integração dos componentes

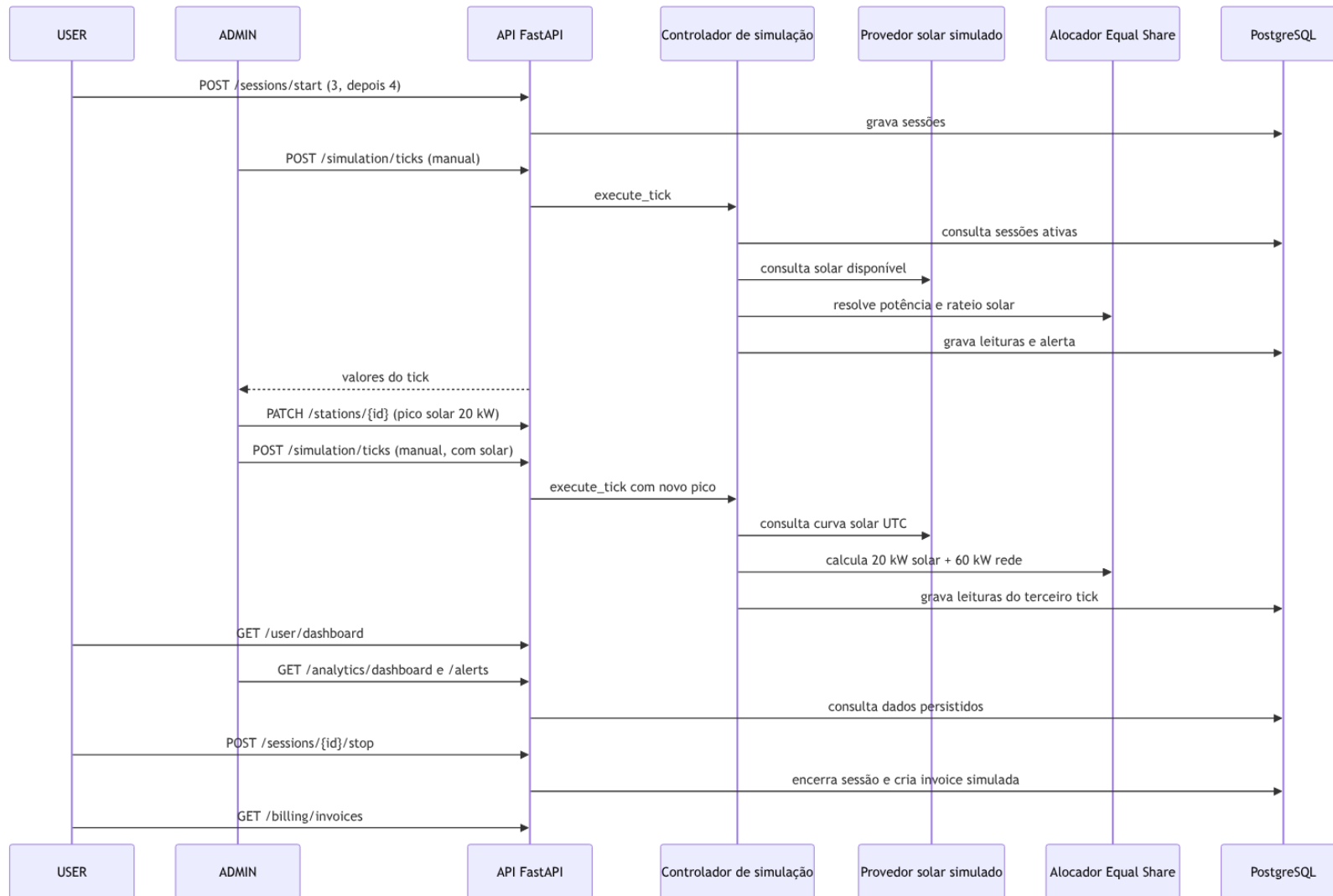
O navegador React consome contratos REST sob **/api/v1**. A API FastAPI coordena serviços de sessão, simulação, energia, billing e analytics. O SQLAlchemy persiste os dados em PostgreSQL. O provedor solar é intercambiável e, nesta sprint, entrega uma curva determinística simulada.



Componente	Responsabilidade	Contribuição
React + TypeScript	Dashboards e interação	Visibilidade operacional e atualização manual
FastAPI + Pydantic	Contratos e orquestração	Automação rastreável com validação
Serviços de domínio	Sessões, alocação, billing e ESG	Regras centralizadas e testáveis
Simulador	Relógio, curva solar e ticks	Ensaio seguro sem hardware
PostgreSQL + SQLAlchemy	Persistência e consultas	Histórico para auditoria e análise

ML é consultivo: previsões futuras nunca poderão violar os limites determinísticos de energia.

Sequência funcional da demonstração



Cada seta corresponde a uma rota, serviço ou persistência existente. O tick é manual; o script apenas encadeia chamadas públicas.

Dashboard administrativo: energia e sustentabilidade

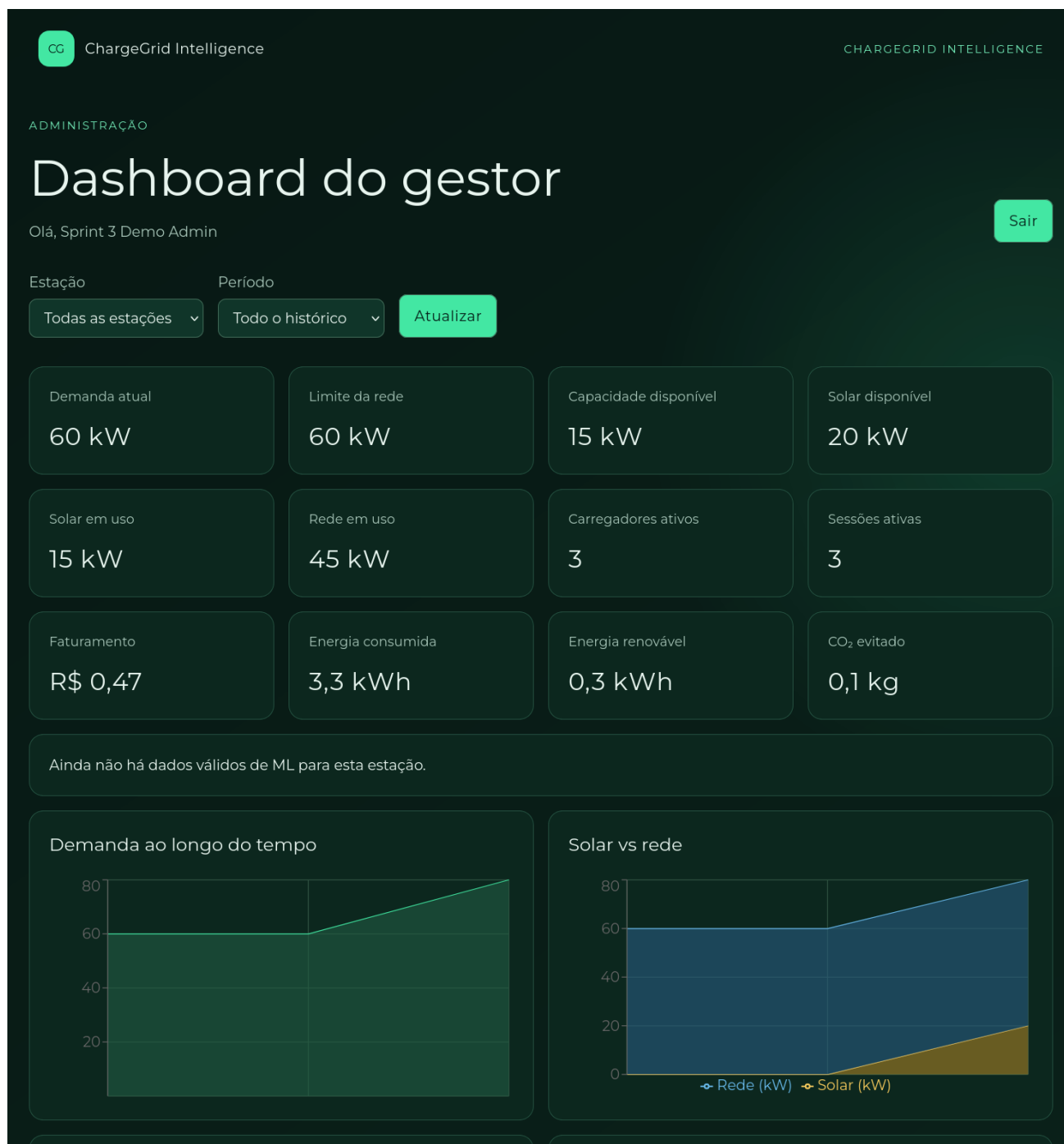


Figura 3. Captura real do dashboard do gestor após a execução do cenário simulado da Sprint 3. A interface consolida demanda, limite da rede, utilização solar, sessões, faturamento e indicadores ambientais.

Após o encerramento da quarta sessão, três recargas permanecem ativas: a demanda total é 60 kW, atendida por 15 kW solares e 45 kW da rede. Os gráficos preservam o histórico do cenário anterior, no qual a entrega chegou a 80 kW com quatro sessões.

Sessões, histórico e alertas

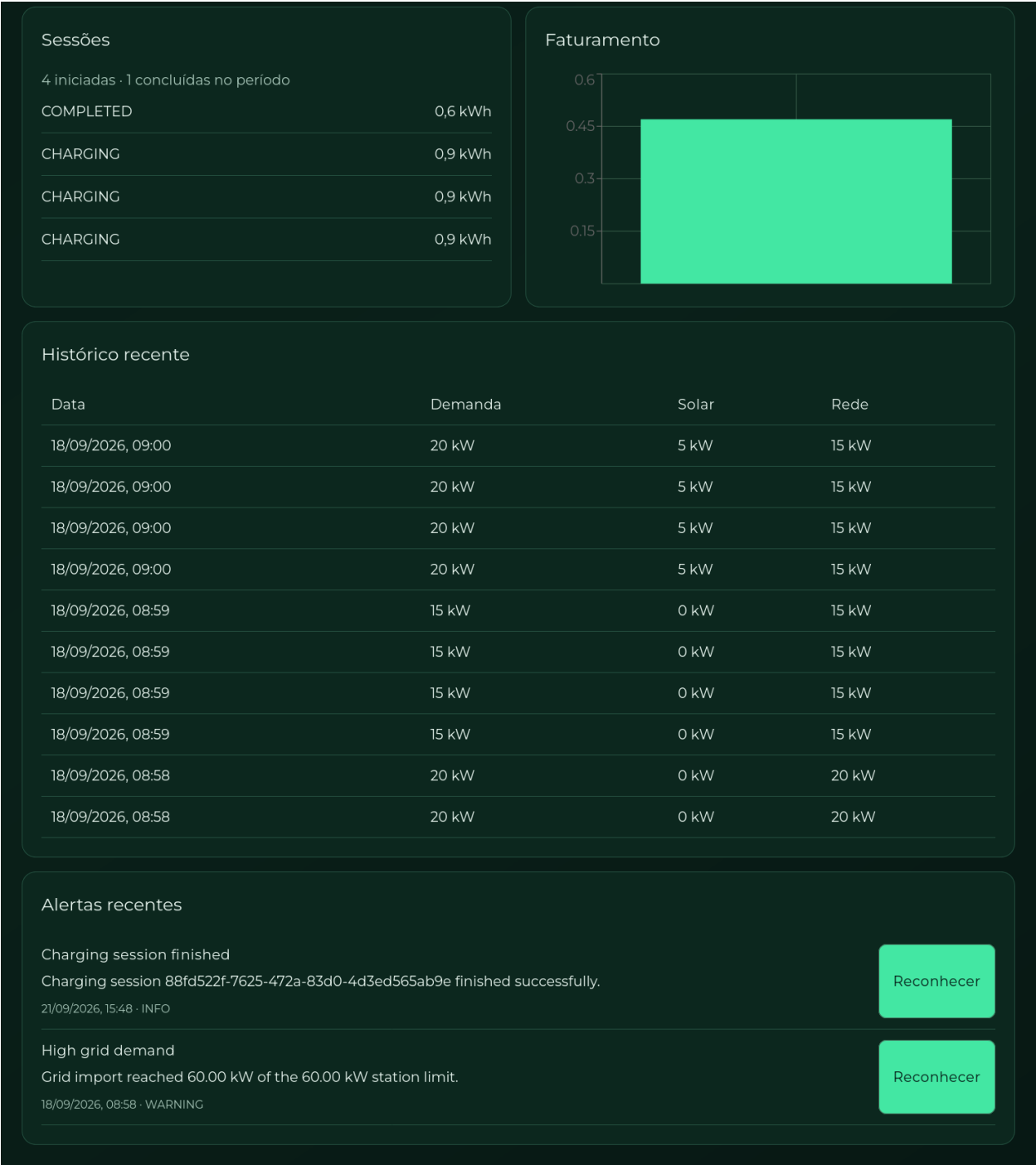
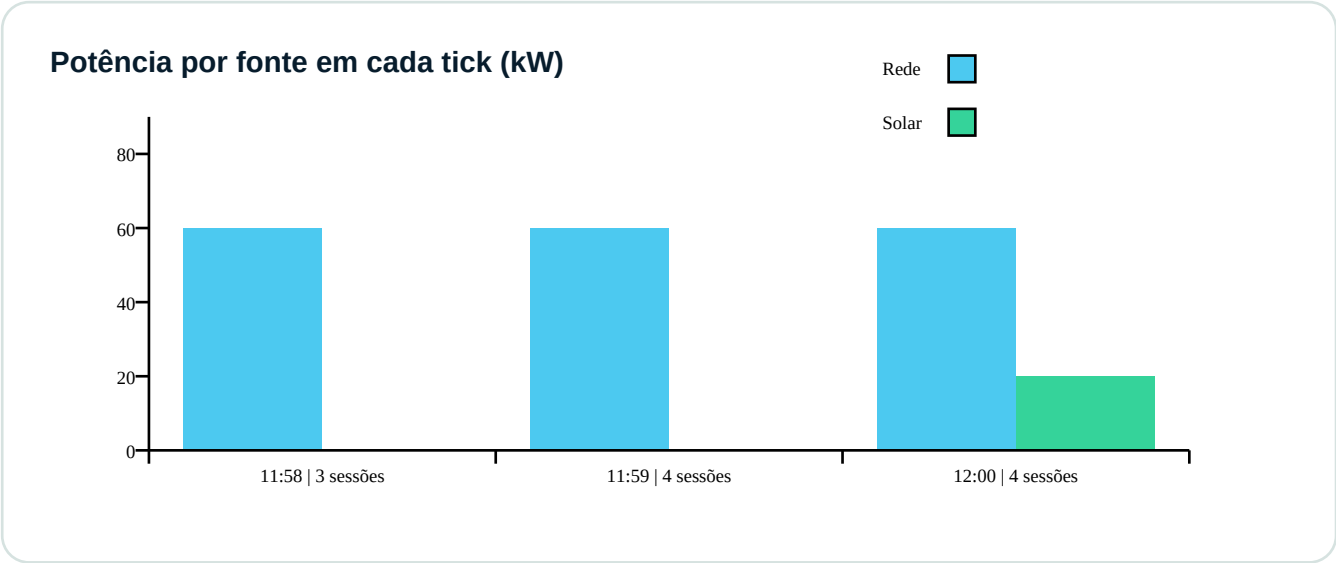


Figura 4. Recorte da mesma captura real, com sessões, faturamento, leituras persistidas e alertas operacionais.

O histórico registra a evolução de 20 kW por sessão para o rateio de 15 kW e, depois, a contribuição solar de 5 kW por sessão. O alerta **High grid demand** confirma que a importação atingiu o limite configurado de 60 kW.

Medições reproduzidas pelo roteiro



Tick UTC	Sessões	Alocação	Solar	Rede	Energia
11:58	3	3 × 20 = 60 kW	0 kW	60 kW	1,0000 kWh
11:59	4	4 × 15 = 60 kW	0 kW	60 kW	1,0000 kWh
12:00	4	4 × 20 = 80 kW	20 kW	60 kW	1,3333 kWh

Indicador	Resultado	Interpretação
Alerta	HIGH_DEMAND	Demanda elevada registrada para decisão do gestor
Invoice	CLOSED - R\$ 0,47	Billing pay-per-use com tarifa de R\$ 0,8000/kWh
CO2 evitado	0,1333 kg	Solar utilizada × fator de 0,4 kg/kWh
Validação	179 + 14 testes	Backend e frontend, além de lint, tipos e build

Escolhas orientadas a correção e demonstração

Escolha	Justificativa
Monólito modular	Reduz infraestrutura e mantém fronteiras claras entre API, domínio, simulação, billing e analytics.
Equal Share	É determinístico, simples de explicar e distribui o recurso escasso igualmente, respeitando limites individuais.
Solar priorizada	Reduz importação da rede e torna explícito o aproveitamento renovável em cada leitura.
Ticks manuais	Permitem repetir o cenário minuto a minuto e inspecionar o efeito de cada decisão.
PostgreSQL + migrations	Garantem persistência estruturada, histórico e evolução de esquema reproduzível.
UTC + Decimal	UTC evita ambiguidade temporal; tipos decimais preservam valores monetários.
API tipada	Pydantic e TypeScript reduzem inconsistências entre backend e frontend.
Simulação desacoplada	EnergyDataProvider permite substituir a fonte simulada por integração física futura sem mover regras críticas.

Invariante central: potência alocada nunca é negativa nem excede pedido, carregador, veículo ou limite da rede.

Como cada tecnologia gera valor

Dimensão	Mecanismo	Efeito demonstrado
Sustentabilidade	Prioridade solar + segregação solar/rede + fator de emissão	20 kW solares no terceiro tick e 0,1333 kg de CO ₂ evitado na API
Automação inteligente	Seed, script HTTP, controlador, alertas e regras determinísticas	Cenário inteiro repetível sem intervenção no banco durante o fluxo
Eficiência energética	Limite de rede e rateio Equal Share	Quatro sessões atendidas sem exceder 60 kW de importação
Eficiência operacional	Dashboards, histórico e billing integrados	Estado energético convertido em alerta, invoice e indicadores

Cadeia de valor

Recarga → dados → informação → inteligência → decisão energética. A plataforma não se limita a registrar consumo: ela relaciona restrição elétrica, fonte energética, evento operacional, custo e impacto ambiental. Essa integração é o núcleo da proposta de valor.

Limites e evolução responsável

O protótipo não comanda potência física. A futura conexão com equipamentos deve entrar pela fronteira do provedor de dados/integração, mantendo os serviços determinísticos como autoridade sobre segurança energética. Um modelo preditivo poderá estimar demanda e classificar risco, mas terá caráter consultivo.

Pensamento Computacional e Automação com Python

Conteúdo	Aplicação no ChargeGrid
Decomposição	Separação em sessões, energia, simulação, billing, analytics e persistência
Abstração	Modelos de usuário, veículo, carregador, estação, leitura e invoice
Algoritmos	Cálculo de potência solicitada, Equal Share, prioridade solar, energia, custo e ESG
Automação em Python	FastAPI, controlador de ticks, seed idempotente e roteiro de demonstração
Estruturas e persistência	Schemas Pydantic, entidades SQLAlchemy e PostgreSQL
Testes e validação	Invariantes energéticas, transições, APIs e fluxo integrado automatizados
Dados e tomada de decisão	Dashboards, alerta HIGH_DEMAND, histórico, cobrança e sustentabilidade

Síntese acadêmica

O projeto materializa o pensamento computacional ao decompor um problema físico e multidimensional em dados, regras e interfaces verificáveis. A automação em Python coordena o experimento, enquanto os testes convertem requisitos energéticos em critérios objetivos. Assim, o software demonstra sustentabilidade e eficiência sem depender de uma alegação de hardware inexistente.

Como verificar a demonstração

1. Configurar ambiente de demonstração isolado, relógio UTC opt-in e fator de emissão de 0,4 kg/kWh.
2. Subir os serviços via Docker Compose e aplicar migrations.
3. Executar o seed idempotente com senhas fornecidas apenas por variáveis de ambiente.
4. Rodar **scripts/sprint3_demo.py**, que usa endpoints públicos.
5. Conferir OpenAPI, dashboards, leituras, alerta, invoice e sustentabilidade.
6. Executar **make check** para validar backend e frontend.

As credenciais permanecem fora do Git; a demonstração deve usar banco limpo e um único controlador de simulação.

Fontes internas consultadas

SPEC.md (fonte de verdade técnica e regras); BRIEFING.md (visão, disciplina e organização); docs/SPRINT_3_PLAN.md; docs/sprint-3/README.md; docs/sprint-3/EVIDENCE.md; diagramas Mermaid/PNG; código e testes do backend/frontend.

Conclusão

A Sprint 3 apresenta um protótipo simulado integrado, rastreável e demonstrável. O sistema mantém a rede em seu limite, redistribui potência, incorpora energia solar, persiste leituras, emite alerta, encerra sessão, calcula cobrança e apresenta indicador ambiental. O resultado conecta automação inteligente, sustentabilidade e eficiência energética em um único fluxo coerente.

CHARGEGRID INTELLIGENCE

Transformando cada recarga em inteligência acionável.